复杂约束下的推箱子（Sokoban）地图程序化生成与验证架构：基于自定义破坏机制与大语言模型空间推理能力的深度分析

引言

推箱子（Sokoban）自诞生以来，凭借其极简的规则集和深不可测的解空间，长期占据着人工智能、路径规划与计算复杂性理论研究的核心地位。从严格的数学和计算复杂度角度进行审视，推箱子问题已经被理论界证明属于 PSPACE-complete 类别 [1]。这意味着随着地图网格尺寸的扩大以及可移动箱子数量的增多，寻找最优解乃至仅仅是验证某个初始状态是否具有可解性，其所需的计算资源和时间都将呈现指数级的爆炸式增长。在经典的推箱子规则框架内，游戏环境中的墙体被设定为绝对的静态障碍物（Static Elements），所有的寻路算法、死锁检测以及启发式评估均建立在这一静态物理拓扑的基础之上。

然而，在标准的推箱子机制之上引入“炸弹（TNT）”或可破坏墙体的机制——即推动特定物体至墙体可引发该墙体及其周围 3x3 区域的静态墙体被永久清空——将从根本上颠覆游戏的状态空间拓扑结构和求解范式。这种机制的引入使得墙体从静态背景转化为动态可变的状态变量，彻底打破了传统启发式搜索中常用的“不可达区域”和“死锁（Deadlock）”预判逻辑。当不可逾越的屏障可以被选择性地消除时，状态树的分支因子将急剧增加，解空间的维度也随之发生非线性扩张。

面对设计三百个尺寸严格限定为 14x10 且必须保证百分之百可解的高质量推箱子地图的需求，涵盖从无墙单箱的基础路径规划到多箱混合炸弹破坏的高难度综合挑战，这已经完全超出了传统人工试错设计的认知和体力极限。与此同时，近年来备受瞩目的大语言模型（LLM）虽然在自然语言处理和代码生成领域展现出惊人的能力，但依赖其直接生成或验证这类具有严格空间逻辑和长视距限制的地图依然存在难以逾越的技术鸿沟。因此，必须构建一套融合“程序化内容生成（Procedural Content Generation, PCG）”、“蒙特卡洛树搜索（MCTS）”、“复杂启发式难度评估”以及“自定义动态规则求解器验证”的自动化工作管线。本研究报告将系统性地深入探讨大语言模型在空间推理中的核心局限性、逆向生成算法的数学与逻辑基础、顶级开源求解器的架构演进，以及针对六个特定难度阶段的系统化地图生成与验证方案。

大语言模型在推箱子空间推理与长期规划中的能力边界

在当前的生成式人工智能领域，大语言模型在零样本或少样本学习中展现出了强大的泛化能力。然而，在探讨是否能够利用这些模型直接设计或求解高质量的推箱子地图时，我们必须对其在空间推理和长期规划任务中的能力边界进行客观、详尽的实证分析。

空间推理基准测试中的性能断崖

近期的多个大型独立评估基准对主流大语言模型在复杂交互和逻辑推理环境中的代理能力进行了极其严格的测试。这些基准测试包括 BALROG、GVGAI-LLM、SokoBench 以及 KORGym 等 [3]。测试结果无一例外地揭示了一个核心现象：大语言模型在应对具有稀疏奖励机制、需要严格物理

碰撞检测以及具备极长规划跨度的网格世界（例如经典的推箱子游戏）时，存在系统性和结构性的缺陷[4]。

为了直观地展示这一性能落差，我们可以对比这些模型在传统强项（如代码生成和数学推理）与严格的空间规划任务（如推箱子基准测试）中的成功率。以下数据汇总了目前业界领先的几种推理模型在不同任务领域的表现差异：

模型名称	代码与数学任务成功率 (均值)	推箱子空间规划成功率 (SokoBench/GVGAI-LLM)
OpenAI o1-preview	85.0%	12.0%
Claude 3.5 Sonnet	82.0%	10.0%
DeepSeek-R1 (强化学习特化)	88.0%	50.0%

数据表明，尽管最新的推理模型在代码生成和复杂数学逻辑上表现出了令人瞩目的优异成绩，甚至在某些基准中逼近或超越了人类专家水平，但在严格的推箱子空间规划测试中，其求解成功率通常呈现断崖式下跌，普遍低于百分之二十。例如，研究指出 OpenAI 的 o1-preview 模型在面对基础的推箱子地图时，成功率仅在百分之十到百分之十二之间徘徊[6]。Claude 3.5 Sonnet 虽然在代码基准测试（如 SWE-Bench）中表现极为出色，但在纯文本或视觉多模态的推箱子任务中，依然无法有效维持长期的空间连贯性[5]。只有极少数采用了特殊的大规模强化学习架构进行长时间探索的模型（如 DeepSeek-R1），在部分规划密集型环境中展现出了一定的潜力，达到了约百分之五十的胜率[4]。然而，即便如此，这距离稳定生成或验证游戏地图所需的百分之百绝对准确率依然相去甚远。

视野崩溃与内部状态追踪的衰退

大语言模型难以胜任推箱子任务的深层原因在于其“内部状态追踪（Internal State Tracking）”机制的灾难性衰退。推箱子游戏具有高度的分支因子，且每一步操作都伴随着极高的风险——将箱子推入角落或靠墙的非目标区域将导致不可逆的死锁。在具体的测试中，模型经常表现出对基本物理规则的无视，例如输出让箱子穿透实体墙壁的指令、将多个箱子推入同一个死角，或者在没有足够后退空间的情况下强行尝试“拉动”箱子[9]。

研究者通过 SokoBench 等工具系统地评估了模型在最小化结构复杂度和极低分支因子下的表现。结果证实，当解决推箱子问题所需的规划步骤超过二十五步时，大语言模型在保持动作序列逻辑连贯性方面会出现严重的崩溃[10]。这种长视距稳定性（Long-horizon Stability）的缺乏，意味着模型无法在没有外部计算环境实时反馈和精确回溯机制的辅助下，仅凭内部的自回归注意力机制来完成全局地图的拓扑推进。在面对如 Phase 4（三箱全局规划）这种需要分解子问题、处理多个死锁陷阱的复杂关卡时，纯语言模型的生成结果有极大概率是结构畸变或存在先天死锁的废图。

基于上述分析，对于一百四十个网格单元（14x10）的地图生成任务，大语言模型的合理应用定位不应是作为直接输出最终地图的生成器，而应作为强大的代码辅助工具。我们应当利用其卓越的编程能力，来编写、调试和优化基于 Python 或 C++ 的程序化生成算法（PCG）、启发式评估函数以及自定义的搜索求解器架构。

保证百分之百可解性的计算基石：逆向生成算法与蒙特卡洛树搜索

在排除了依赖大语言模型直接生成地图的路径后，我们必须全面转向计算机科学中经过严格数学验证的程序化内容生成（Procedural Content Generation）技术。由于推箱子的状态空间极其庞大，采用完全随机生成墙壁和箱子然后再调用求解器去验证其是否可解的方法（即正向随机生成法）效率极其低下。统计学和实验证据表明，随着棋盘复杂度的上升，随机生成的地图有超过百分之九十九的概率是先天死锁或不可解的，这会导致计算资源的巨大浪费 [2]。

游戏机制逆转与程序倒置

为了在算法层面上从根本上确保所生成的三百个地图中每一个都具备百分之百的可解性，学术界和工业界公认的最佳实践是采用“逆向生成法（Reverse Play / Game Mechanic Inversion）” [11]。

该算法的核心架构基于状态空间的反向探索。其初始设置并非游戏开始时的状态，而是一个明确定义的“胜利状态（Goal State）”。在胜利状态下，预设数量的所有箱子都已经被准确无误地放置在预设的目标点上。此时，生成器接管玩家角色的控制权，在地图中执行随机或受引导的“拉动（Pull）”操作，而不是标准正向游戏中的“推动（Push）”操作 [13]。

在逆向运行的时间线中，游戏的物理规则必须发生严格的反转：

第一，玩家角色必须处于目标箱子的相邻正交网格。

第二，玩家角色沿着远离箱子的方向向后移动一步，同时将箱子“拉”向玩家原本所在的网格位置。

第三，执行拉动操作的前提是，玩家角色后退的目标网格上必须是空地，不能有静态墙壁或其他箱子的阻挡。

通过这种严谨的程序倒置（Program Inversion）技术进行的状态扩展，其在图论和数学上完全等价于在正向时间线中进行了一次从当前状态走向胜利状态的有效推动 [13]。因此，无论生成器在网格中执行了十次、五十次还是一百次合法的逆向拉动，只要算法在任意时刻停止执行，当前整个棋盘的布局状态就必然存在一条（至少一条）严格通往胜利状态的解路径。这种方法在算法设计层面直接规避了生成不可解地图的风险，使得生成器的产出始终处于有效状态空间的子集内。

结合蒙特卡洛树搜索（MCTS）进行质量与难度调控

然而，仅仅保证地图的可解性并不等同于生成了“高质量”的谜题。如果生成器只是在地图中进行纯粹无目的的随机随机拉动，最终产生的地图布局往往显得非常琐碎、缺乏逻辑深度，或者玩家

只需通过简单的直线推动即可过关。为了生成满足不同阶段（尤其是如 Phase 3 的互挡预判和 Phase 4 的全局规划需求）的高质量地图，必须将生成过程建立在一个强大的搜索算法之上。蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）已被证明是解决这一问题的理想框架 [15]。

在将 MCTS 应用于推箱子地图生成的架构中，搜索树的每一个节点代表一个具体的棋盘状态配置，而树的边（Edges）则代表合法的逆向“拉动”动作。MCTS 算法通过反复执行选择（Selection）、扩展（Expansion）、模拟（Simulation/Rollouts）和反向传播（Backpropagation）四个阶段，在巨大的可能布局空间中进行定向探索。在模拟阶段，算法会快速且随机地执行一系列拉动动作，生成一个候选的完整地图布局。

为了引导 MCTS 算法向着“困难”且“有趣”的方向进化，必须定义一个基于数据驱动的评估函数（Evaluation Function）。该函数负责对模拟生成的候选地图进行质量打分，其得分将反向传播更新搜索树，从而影响后续节点的选择概率 [18]。

人类认知视角的地图质量与难度评估指标

高质量的推箱子地图评估不能仅仅依赖于路径的绝对长度，因为这并不能完全反映人类玩家在解题时所面临的认知挑战。评估函数的构建必须融合多维度的启发式指标，这些指标需要与人类感知难度和认知负荷高度相关 [12]。

评估体系通常包含以下几个核心计算维度：

第一是空间拥挤度与依赖关系（Congestion and Precedence Constraints）。拥挤度指标通过计算不同箱子在移动路径上的重叠区域和相交频率来衡量地图的紧凑性 [16]。高度的路径重叠在正向游戏中意味着资源的争夺，玩家必须遵循极其严格的先后顺序才能避免死锁。例如，必须先将某个箱子推入一条狭窄走廊的深处（即停车 Parking操作），腾出关键节点，才能让另一个箱子通过 [20]。这一指标对于生成 Phase 3（两箱互挡）和 Phase 6（结构紧凑高难）的地图起着决定性的导向作用。

第二是箱子变轨数与工作记忆负荷（Box Changes and Working Memory）。单纯的步数（Moves）和推动数（Pushes）虽然容易计算，但往往不足以衡量真正的难度 [12]。箱子变轨数（Box Changes）专门记录玩家在最优解序列中，停止推动当前箱子并转而前往地图另一端推动另一个箱子的频率 [1]。每一次变轨都要求玩家在大脑中重新加载另一个局部区域的空间状态，这极大地增加了人类认知过程中的工作记忆（Working Memory）负担 [19]。研究表明，该指标与人类玩家解开谜题所需的时间呈现极高的统计学正相关性。

第三是空间拓扑的同质性惩罚与 3x3 墙体簇（3x3 Blocks）。一个优秀的推箱子地图通常需要复杂的几何地形来限制移动自由度。评估函数会奖励那些存在较大实心墙体（如 3x3 墙体簇）的地图配置，同时惩罚那些存在大片连续开阔空地的布局，以此来强制生成器制造出深邃的迷宫结构和瓶颈点 [18]。

第四是问题分解与反直觉移动（Problem Decomposition and Counterintuitive Moves）。人类在解决复杂问题时高度依赖将大问题分解为若干个低耦合的子问题 [19]。高难度的推箱子地图往往具有

极高的“谜题熵（Puzzle Entropy）”，其特点是破坏玩家直觉。反直觉移动指标专门计算在已知的最短解路径中，箱子被迫向着原理目标点方向移动的步数总和[22]。这种“欲进先退”的设计是优秀推箱子关卡的标志。

通过在 MCTS 算法中对这些指标进行加权组合，形成复合评估函数，系统能够在逆向生成的过程中自动筛选和进化，最终产出在结构和逻辑上都极具挑战性的高质量初始状态。

自定义规则的挑战：TNT 炸弹机制与动态状态空间架构

在用户的设计需求中，最为特殊和打破常规的是引入了“炸弹（TNT）”机制（主要应用于 Phase 5 和 Phase 6）。该机制规定，玩家可以将带有炸弹属性的特殊物体推动至与墙体相邻的位置，碰撞触发后将清空以撞击点为中心的 3x3 区域内的所有墙壁。这种动态破坏机制在经典的推箱子求解器和生成框架中是完全不存在的，它的加入彻底重构了游戏的底层数学模型。

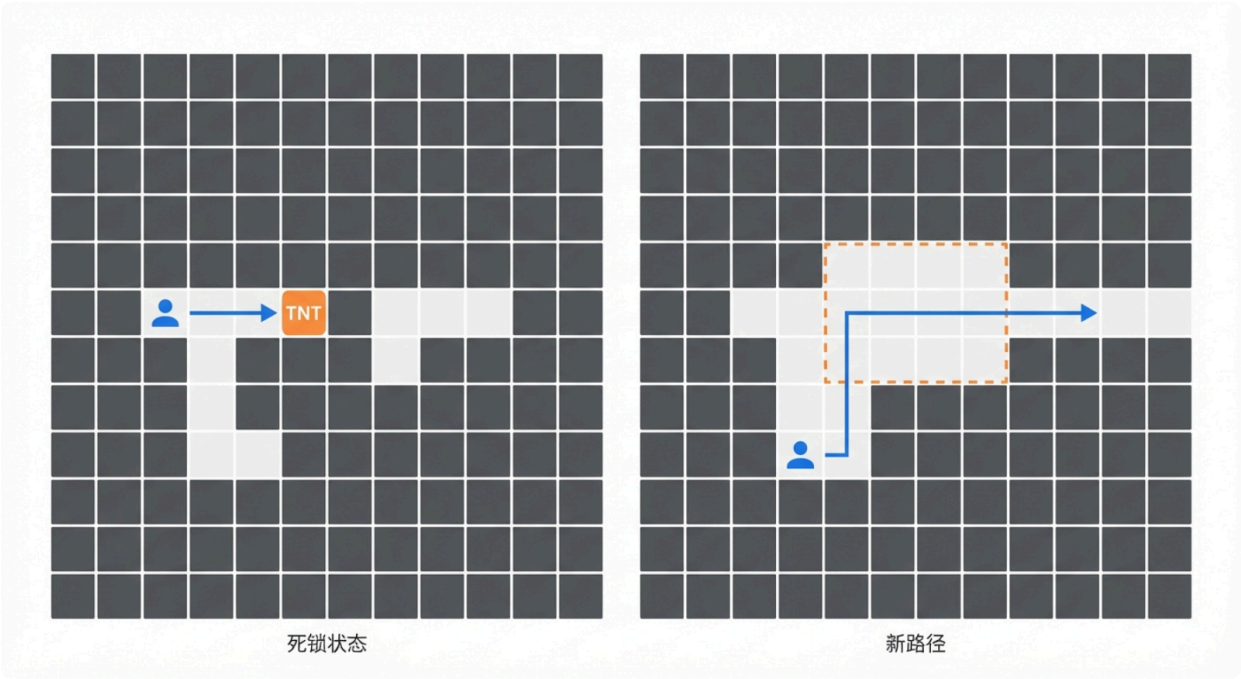
状态表示的维度扩展

在分析世界顶级的开源推箱子求解器（例如基于 FESS 搜索算法的 Festival，或者基于 A* 算法的 YASS 和 Sokolution）时，可以发现它们实现极速搜索的一个根本前提是：墙体是被视为永不改变的静态全局位图（Static Bitboard）或预计算的查找表[23]。这意味着求解器在初始化阶段就可以一次性计算出所有房间的连通性、所有绝对的死角（Dead ends），并建立不可变的模式数据库（Pattern Databases）。

然而，一旦引入 TNT 机制，墙体就不可避免地变成了动态状态空间的一部分。假设传统的推箱子状态可以精简表示为 $S = \{P_x, P_y, B_1, B_2 ... B_n\}$，其中 P 代表玩家的归一化坐标，B 代表各个箱子的坐标。在加入炸弹元素后，整个系统的状态定义必须扩展为 $S' = \{P_x, P_y, B_1 ... B_n, T_1 ... T_m, W_{bitmap}\}$，其中 T 为炸弹的实时坐标，而 W_{bitmap} 则是当前地图所有墙体是否存在的一维或二维位图状态。

这种扩展带来了严峻的计算挑战。墙体可能在任意时刻被破坏，使得原本不可达的闭塞空间瞬间变为可达区域。这直接导致用于防止重复搜索状态的哈希转置表（Transposition Table）必须进行根本性重构。在计算 Zobrist Hash 键值时，不能仅仅对箱子和玩家位置进行异或（XOR）运算，还必须将庞大的墙体状态矩阵也进行哈希编码纳入其中[24]。这种空间维度的非线性扩张对求解器的内存管理和搜索效率提出了极高的要求。

TNT 机制引发的状态树跃迁与死锁破除



引入TNT后，墙体从静态障碍物转变为动态状态变量。左侧：传统视角下的绝对死锁状态（箱子被完全封死）。右侧：TNT推入墙体引发 3x3 爆破后，物理屏障被清除，图结构发生改变，原本不可达的节点被重新纳入 A* 搜索的开放列表（Open List）中。

启发式函数的重新定义与动态寻路

为了让改造后的自定义求解器能够快速且准确地验证 Phase 5 和 Phase 6 的复杂地图，传统的 A* 算法中的启发式函数（Heuristic Function）必须进行相应的理论修正。

在标准的推箱子求解器中，通常采用“二分图匹配下的最小曼哈顿距离（Bipartite matching of Manhattan distances）”作为状态到目标距离的启发式估计值（即 h-score）[25]。但在墙壁可破坏的规则下，如果仍然沿用静态墙壁的障碍假设，原有的距离计算往往会大幅度高估（Overestimate）实际需要的移动步数。在 A* 搜索理论中，一旦启发式函数高估了成本，算法就会失去其核心的“可采纳性（Admissibility）”，进而无法保证最终找到的解决方案是最优的，甚至可能导致搜索路径的偏离[26]。

针对这一理论缺陷，必须引入**动态寻路启发式（Dynamic Pathfinding Heuristic）**架构。在这种新架构下，底层图结构的连通性评估不仅要计算现有的空地，还要预判潜在的爆破空间。在计算任意箱子到目标点的启发式距离时，算法应当将常规的静态墙壁视为成本无限大的不可逾越节点，但对于“可通过炸弹破坏的墙壁区域”，则应将其视为具有较高惩罚权重（例如，其权重等于将炸弹推至引爆点所需的最小步数加上通过破坏区域的基准成本）的可通行节点。这种动态权重的分配使得启发式函数能够“穿透”那些即将被摧毁的墙壁来评估状态潜力，从而保证搜索算法在复杂

动态环境中的高效收敛。

面向 14x10 尺寸的顶级数据集资源与预处理

在着手构建六个特定难度的地图生成流水线之前，引入高质量的外部数据集作为生成算法的“种子（Seeds）”或结构参考，能够显著提升程序化生成内容的起始质量，避免算法从绝对的白板状态进行毫无章法的盲目搜索。

核心开源数据集评估

目前在学术界和开源社区中，存在几个极具价值的推箱子标准化地图数据集，它们为不同阶段的生成策略提供了丰富的设计组件和验证基准：

第一，**Boxoban Dataset (Google DeepMind)**。这是一个由 DeepMind 专门为训练无模型深度强化学习代理而发布的大型开源数据集，包含了多达一百万个由程序化算法生成的地图配置 [27]。该数据集按照难度被严格划分为“未过滤（Unfiltered）”、“中等（Medium）”和“困难（Hard）”三个级别 [27]。其原始配置尺寸统一为 10x10，格式采用标准的 ASCII 字符编码（例如 # 代表墙壁，@ 代表玩家，$ 代表箱子，. 代表目标点）[27]。虽然尺寸略小于目标需求，但其海量的高难度布局逻辑可以被轻易提取。

第二，**Microban Dataset**。由著名推箱子设计师 David W. Skinner 创作的 155 个关卡合集 [27]。这套数据集在推箱子社区享有盛誉，它以尺寸小巧紧凑且极其富含“特定教学机制（particular concept）”而著称 [28]。它不追求无意义的地图广度，而是将经典的几何死锁陷阱（如互挡、必经之路的资源争夺）浓缩在有限的空间内。这非常符合我们对高质量地图的定义。

第三，**XSokoban** 与 **Sharpened Levels** 体系。XSokoban 是一个包含 90 个经典困难关卡的标准化测试基准，常用于衡量现代求解器的极限性能 [29]。在此基础上，推箱子社区演化出了“Sharpened Levels（锐化关卡）”的设计哲学。如 DrFogh 等设计师通过“填塞无用空地（stoning unneeded floors）”、压缩地图边界、增加目标箱子数量等微调手段，在不改变原有核心拓扑的前提下，极大地提升了地图的解题步数和逻辑复杂度 [16]。这种“锐化”理念完美契合了我们要设计的 Phase 6（结构紧凑高难）的核心思路。

尺寸适配与格式转换工程

考虑到 Boxoban 数据集的原始尺寸为 10x10，为了将其无缝集成到我们的 14x10 目标生成管线中，需要进行空间维度的预处理。这不仅是一个简单的尺寸放大问题，更关乎游戏逻辑空间的合理布局。

可以通过编写自动化脚本读取标准 ASCII 格式的地图矩阵，并执行中心对齐包裹（Padding）操作。具体而言，将 10x10 的功能性核心结构置于 14x10 网格的正中央，然后在外部边缘对称地填充宽度为两格的 # 墙体区域。这种包裹处理不仅将尺寸标准化，还为后续的“炸弹机制”提供了充足的“可破坏物质”，使得生成器能够在不破坏内部核心死锁逻辑的前提下，在外部厚重的墙体上做文章，打通原本不存在的捷径。

此外，由于顶级开源求解器和验证工具（如 YASC、JSoko）普遍支持 XSB 或 LURD 等标准化记录格式，在整个自动化流水线的数据流转过程中，保持这种字符映射的严格一致性是保证各个子模块顺利衔接的技术保障[30]。

顶级开源求解器的技术架构剖析与改造实施方案

为了支撑生成算法中至关重要的“地图可解性严格验证”环节，特别是为了处理引入动态墙体和 TNT 后的复杂空间计算，我们必须深入剖析现有顶级求解器的底层机制，并制定出切实可行的改造和实施路径。

开源求解器性能与算法逻辑一览

纵观当今性能最强悍的推箱子开源求解器矩阵，它们无一例外地采用了深度优化的 C++ 语言编写，并将高级图搜索算法与极其丰富的领域特定知识（Domain-specific Knowledge）进行了深度耦合：

- **Festival (基于 FESS 算法)**：由 Yaron Shoham 研发的 Festival 是当前学术界和应用领域公认性能最强的求解器之一[23]。它不仅是首个能够完美求解全部 90 个经典 XSokoban 极限基准测试关卡的程序，其核心突破在于采用了创新的 FESS（Fast Evolutionary State Search）算法[23]。该求解器依赖于高度复杂的模式数据库（Pattern Database Heuristics）来提供极其精确的下界距离估计，从而能够在极短时间内跨越漫长的搜索深度。
- **YASS (Yet Another Sokoban Solver)**：由 Brian Damgaard 倾力打造，是另一个在该领域具有统治地位的开源工具[32]。YASS 的强大在于其无可匹敌的静态死锁检测和剪枝技术。它在其 A* 搜索主干中集成了如 PI-Corral 剪枝（PI-Corral pruning）、隧道方块识别（Tunnel Squares）以及目标室逆向填充预判（Goal room packing）等高级算法机制[25]。这些机制使得 YASS 能够在其搜索树展开的极早期就识别出潜在的无效状态并迅速将其截断。
- **Sokolution**：由 Florent Diedler 开发，该求解器在设计哲学上另辟蹊径。它放弃了对超大尺寸地图的支持，通过严格限制地图中空地方块的总数不得超过 256 个，从而能够将所有的坐标和状态压缩至单字节（Byte）的底层位集（Bitsets）数据结构中[24]。这种极致的内存布局优化赋予了 Sokolution 无与伦比的计算吞吐量和运行速度，使其非常适合嵌入到需要海量并发验证的自动化流水线中。
- **Rolling Stone**：尽管年代较为久远，由阿尔伯塔大学开发的 Rolling Stone 依然是自动化求解领域的先驱[33]。它奠定了使用迭代加深 A*（IDA*）作为核心搜索策略的基础，并证明了将通用搜索算法与二分图匹配等领域增强技术结合的有效性。

针对动态炸弹机制的求解器架构改造

试图直接在这类顶级 C++ 求解器（如 Festival 或 YASS）的源码中强行植入“墙体可瞬间消失 3x3 范围”的炸弹逻辑，在工程实践上是极其危险且不切实际的。这些顶级程序的死锁识别模板和启发式加速逻辑是深度硬编码（Hardcoded）在“墙壁永不改变”这一绝对真理之上的。强行改变墙壁状态，将导致底层的 PI-Corral 剪枝机制失效，精心设计的位图哈希表发生大规模冲突甚至崩溃，最终使求解器的性能退化为盲目的穷举搜索。

因此，为了确保自动化工作流的稳健性和开发的敏捷性，针对包含 TNT 的 Phase 5 和 Phase 6 的

可解性验证，强烈建议采用基于高层编程语言（如 **Python** 或现代化 **C++**）的基础 **A*** 或广度优先搜索（**BFS**）开源框架作为改造基底 [34]。

具体的算法改造和实施步骤如下：

1. 选择基础框架：获取如 GitHub 上开源的 ba-sokoban-solver（基于 Python 3.10+，支持 BFS、Dijkstra、A*、IDA* 的模块化求解器）或类似的支持面向对象设计的状态搜索库 [35]。
2. 动作空间的扩展：在控制逻辑模块的状态扩展（State Expansion / Transition）函数中，除了原有的向上、下、左、右推动和移动逻辑外，增加全新的条件分支：评估玩家角色试图推动的目标对象是否为特殊的 BOMB 实体，并且该实体移动方向的下一个相邻网格是否为 WALL。
3. 状态转换与矩阵变异：如果上述前置条件成立，则触发爆破逻辑。此时不能仅仅更新物体坐标，必须对整个地图的状态矩阵应用位遮罩（Matrix Mask）操作。算法将以爆炸中心坐标 (x, y) 为锚点，通过边界检查后，将其周围九宫格内的所有坐标集合 $(x \pm 1, y \pm 1)$ 的状态标志强制重置为 Floor（空地）。
4. 动态节点入队：经过拓扑重构的新地图矩阵副本，将作为一个全新的复合状态节点，附带更新后的动态寻路启发式评分，被重新压入 A* 算法的优先级队列（Priority Queue）中继续参与搜索迭代。

尽管相比于极致优化的底层 C++ 求解器，这种高级语言架构在面对步数极长的传统巨型地图时性能存在劣势，但考虑到用户需求的地图尺寸严格限定为较小的 14x10 网格，经过哈希函数精简优化后的定制求解器在现代处理器上依然可以保证在合理的时间窗口（如数秒至几分钟）内完成确定性的验证。这完全能够满足离线生成和筛选三百张目标地图的工程性能预算。

六大阶段地图流水线：从基础路径到极致复合挑战的设计与生成

有了上述涵盖人工智能分析、逆向搜索算法、评估指标体系以及自定义求解器验证的完整计算理论基石，我们现在可以系统地针对用户提出的六个难度阶段，分别设计出一套可自动化执行、具备高度容错率，且能百分之百保证产出高质量 14x10 规格地图的实施方案。

Phase 1 (1箱无墙)：开阔空间与长距离路径规划

设计理念与评估目标：

本阶段旨在测试初学者对基础位移逻辑和长视距无障碍路线规划的掌握。地图中没有任何可能导致死锁的内部静态障碍物，挑战完全来自于箱子起始位置与最终目标点之间不断增加的绝对距离跨度。

程序化生成流水线：

1. 矩阵初始化：在内存中初始化一个 14x10 的全空状态网格。通过遍历矩阵边缘，仅在四周外围保留一圈用于封闭空间的边界墙（Bounding Walls），内部区域全被标记为开阔的空地（Floor）。
2. 胜利状态锚定：在网格的内部区域完全随机地选取一个坐标点放置唯一的目标点（Target），

并将对应的箱子（Box）初始放置在该目标之上，形成逆向算法所需的“已解决状态”。玩家角色被随机分配在箱子的正交相邻空地上。

3. 约束下的逆向随机漫步（**Constrained Reverse Random Walk**）：启动逆向拉动算法。由于内部空间毫无阻挡，纯粹的无定向随机拉动极易导致箱子在目标点附近进行无意义的局部徘徊。因此，必须引入距离引导机制。
4. 距离最大化驱动：在每一次执行逆向拉动动作后，算法实时计算箱子当前所处位置与初始目标点之间的欧几里得距离或曼哈顿距离。系统在候选的动作树分支中施加强烈的偏置概率，迫使算法优先选择能够使箱子在物理空间上显著远离目标点的拉动动作。
5. 阈值筛选与入库：迭代运行该生成循环数千次产生候选池。设置严格的距离准入阈值（例如，要求箱子初始位置与目标位置的绝对曼哈顿距离必须 ≥ 15），从而筛选出符合长距离位移要求的首批 50 个高标准地图作为最终输出。

Phase 2 (1箱有墙)：内墙分割与复杂迷宫制造

设计理念与评估目标：

通过引入大量的内部静态几何障碍物，打破单一的开阔视野，迫使玩家无法直线到达目标，必须进行多次绕路、寻找转角通道以及规避视觉盲区带来的局部死胡同。

程序化生成流水线：

1. 拓扑骨架构建：这一阶段的底层基础不再是空白网格，而是迷宫。采用基于图论的迷宫生成算法，如随机化深度优先搜索（Randomized DFS）或细胞自动机（Cellular Automata）算法，在 14x10 的边界内部生成连绵曲折的墙体分布 [12]。在生成过程中，算法必须确保迷宫的图连通性，避免出现大面积与主干道物理隔离的封闭死区。
2. 逆向拉动与状态膨胀：在生成的迷宫拓扑上，随机放置一个目标并启动逆向拉动机制。墙壁的加入自然限制了拉动的方向，使得生成的路径天生具备“绕路”属性。
3. 曲折度（**Tortuosity**）深度评估：质量控制的核心在于衡量绕路的复杂性。利用基础 A* 算法正向计算生成的地图中箱子通往目标的最短解路径。遍历该路径的向量序列，提取出“方向改变次数（Number of Turns）”作为难度评估的决定性指标。
4. 排序与提取：在候选地图集合中进行降序排列，筛选出转折次数最多、迫使玩家在迷宫各个走廊中进行多次反复“折返跑”的 50 个地图。

Phase 3 (2箱有墙)：资源争夺与多智能体推演预判

设计理念与评估目标：

引入第二个箱子将游戏的逻辑维度提升至真正的资源博弈层面。玩家不仅需要考虑单一路径的连通性，还必须严密计算两个箱子在狭窄通道内的推送先后顺序。轻率的决策将导致一个箱子将另一个箱子的退路彻底堵死（即典型的局部互挡死锁）。

程序化生成流水线：

1. 多目标状态设定：在预先生成迷宫墙体拓扑的底图上，放置 2 个物理分散的目标点和 2 个对应的箱子。

2. 多智能体逆向交替：算法的控制逻辑升级。在进行状态空间的逆向拉动时，算法模拟玩家角色在网格中穿梭，交替拉动这两个不同的箱子。这确保了即便有两个箱子共同存在，整个系统历史的连贯性依然指向一个统一的可解根节点。
3. 拥挤度与依赖关系（**Precedence Constraints**）的 **MCTS** 注入：此时，蒙特卡洛树搜索（MCTS）开始发挥其强大的空间导向作用。在 MCTS 的综合评估函数体系中，算法调度器将大幅度上调“拥挤度（Congestion）”指标的计算权重 [18]。
4. 空间冲突的高额奖励：评估系统会极大地奖励那些在树搜索过程中，出现“两个箱子的逆向拉动路径在极为狭窄的走廊或唯一通道中发生高频次空间重叠”的候选分支节点。在时间线逆转的正向逻辑中，这种刻意制造的路径高度重叠直接等价于：玩家在行进到此节点时，必须先将箱子 A 艰难地推入指定深处（停车 Parking 操作），随后才能勉强腾出空间让箱子 B 通过关键的物理瓶颈。这种需要多步预判的互挡逻辑正是推箱子进阶魅力的核心体现 [20]。

Phase 4 (3箱复杂墙)：空间解构与全局视野宏观规划

设计理念与评估目标：

要求玩家具备统揽全局的宏观视野。玩家必须应对横跨整个地图的多个隐蔽死锁陷阱，并且需要在不同的地理分区之间来回调度三个箱子，考验其长周期的空间记忆和规划能力。

程序化生成流水线：

1. 二叉空间分割（**BSP**）与房间拓扑：针对 14x10 的规格，单纯的迷宫已不足以支撑宏观规划。算法采用二叉空间分割（BSP Tree）技术，将整个网格物理划分为 3 到 4 个相互独立却又通过狭窄咽喉（宽度为 1 或 2 的“门”或“隧道/Tunnels”）相连的独立房间 [25]。
2. 目标与资源的极端分离：强化空间的跨度感。算法将 3 个目标集中聚拢放置在地图一侧的某一个房间内（定义为目标室），而将初始逆向拉动的终点（即正向游戏时箱子的起点）尽可能地逼近至距离目标室在图论意义上最远的边缘房间中。
3. 宏观问题解构（**Problem Decomposition**）的难度标定：基于认知心理学界关于人类解决推箱子问题依赖潜意识问题分解机制的研究结论 [19]，系统在 MCTS 中设置刚性的结构筛选条件。
4. 强制的跨区调度：生成引擎强制规定，任意一个箱子从初始产生点到最终目标点的理论最短路径，必须在拓扑结构上穿过至少两个不同的“门”。这种严苛的设计强制玩家不能再采用走一步看一步的局部策略，而必须站在全局维度上规划三个箱子穿越多重关卡的宏观先后顺序，从而产生极高的认知挑战。

Phase 5 (3箱+TNT)：构造性前向破坏与捷径开辟验证

设计理念与评估目标：

引入颠覆性的物理破坏机制。玩家需要通过逻辑推理，准确识别出地图中封锁关键路径的承重墙体，并精确调度引爆点，利用有限的炸弹进行爆破以打通闭塞的捷径。

程序化生成流水线（摒弃逆向，采用前向构造与严苛验证）：

正如前文深入探讨的，由于炸弹引发的墙体崩塌在逆向时间轴中难以通过简单的启发式规则来进

行合理的物理还原模拟，本阶段必须彻底切换为**“构造性前向生成与定制求解器验证（Constructive Forward Generation with Solver Validation）”**的全新开发范式。

1. 基础地貌继承与闭环封锁（**Path Blocking**）：首先提取 Phase 4 产出的具有多房间复杂拓扑结构的优秀地图底稿。算法运行连通性分析，找出箱子到达目标室的关键必经通道，并刻意生成实心墙体将其彻底物理堵死。此时，传统的求解器必然会判定该地图处于绝对的无解（Unsolvable）状态。
2. 破坏性资源的战略放置：在远离封闭通道的其他迷宫分支末端或盲区，放置 1 到 2 个具备破坏属性的 TNT 炸弹。
3. 双重反证法验证机制：调用经过特殊定制、完全支持动态矩阵变异和状态扩展的 Python/C++ A* 求解器（详见前文技术架构章节 ）。验证程序在后台无头执行，必须同时证明并满足两个严苛的命题：
   - 命题 **A**（死锁证明）：如果在整个搜索树的展开过程中，动作空间不包含任何引爆炸弹的操作，求解器的开放列表（Open List）最终必然被清空，返回确切的无解证明（系统死锁 ）。
   - 命题 **B**（破局证明）：只有当在特定的时间和确切的坐标将炸弹推入阻碍的墙体引发 3x3 爆破后，地图的连通图结构发生重组，求解器才能够成功找到一条抵达胜利状态的完整路径。
4. 只有同时完美通过这两项理论验证测试的地图，才会被系统标记为合格的 Phase 5 成品，累计输出 50 种符合条件的独特布局。

Phase 6 (混合究极高难)：紧凑拓扑下的机制深度耦合与极限微操

设计理念与评估目标：

这是推箱子挑战的终极形态。在 14x10 的极其狭小且受限的物理空间内，高密度地集合了多箱调度、复杂墙体死锁以及炸弹破坏机制。通关需要大量的冗余步骤、极度复杂的状态置换以及毫无差错的极限逻辑微操。

程序化生成流水线：

1. 多目标进化算法（**MOEAs**）的引入：面对如此高维度的机制耦合，单一的搜索或验证已经难以寻找到设计空间中的最优解。算法架构升级为部署多目标进化算法（Multi-objective Evolutionary Algorithms, MOEAs），例如最新学术文献中详细论证的改进型双存档算法（Two_Arch2），来逐步逼近地图设计的帕累托最优前沿（Pareto Front）[38]。
2. 染色体建模与遗传算子：将 14x10 的地图抽象建模为包含特定整数编码的二维数组染色体（例如 0=绝对空地，1=不可破坏墙，2=箱子，3=目标点，4=TNT 炸弹，5=可破坏墙体边界）。在庞大的种群中持续进行交叉（Crossover）、变异（Mutation）以及优胜劣汰的遗传操作。
3. 多维优化目标的联合驱动：
   - 优化目标 **1**（极致空间效率）：极力最小化地图中所有的“未使用空地（Unused Floors）”和无效缓冲带数量 [40]。强行压缩系统的熵，迫使整个环境结构极其紧凑，制造出牵一发而动全身的空间挪移约束。
   - 优化目标 **2**（解路径深度）：在保证验证求解器返回可解的前提下，最大化完成关卡所需的绝对最小通关步数。
   - 优化目标 **3**（机制的深度耦合度）：地图状态机必须强制交织两种机制：在解题的中间阶

段，必须发生“为了获取被重重封锁的 TNT，需要利用多个箱子进行复杂的互挡调度来铺路”，以及随后“利用获得的 TNT 消除特定的承重墙壁，从而释放原本深陷死局的关键目标箱子”的复合递归逻辑。

4. 算力提纯：让这套庞大的进化算法阵列在高性能服务器集群上持续迭代运行数百万个代次，最终从趋于收敛的高维解空间中，精确提取出 50 种处于帕累托最优状态（即在空间最为紧凑的同时保证了最冗长的逻辑解题步数）的 14x10 究极组合。

结论

综上所述，为了满足在 14x10 的受限网格环境中，系统性地设计出三百个难度呈阶梯式递进、且深度融合了创新型 TNT 炸弹破坏机制的高质量推箱子地图的需求，试图单纯依靠人类设计师的主观经验法则，亦或是寄希望于当前大语言模型在零样本下的空间逻辑推理能力，都是在理论和工程实践上被证明行不通的路线。

唯有跨越这些局限，构建起一套以计算和搜索为核心的主导范式，方能破解这一难题。这套范式必须紧密集结基于底层机制倒置的**“逆向拉动算法”来确立可解性的物理基石，引入“蒙特卡洛树搜索（MCTS）”并在其中注入多维度的人类认知难度评估模型，进而在面对打破传统的爆炸机制时，果断切换至基于高级编程架构深度改造的“动态状态空间 A* 求解器”执行极其严苛的双重反证前向验证，最后在复杂机制耦合的顶端运用“多目标进化算法（MOEAs）”**进行解空间的极限提纯。运用这种高度工程化、算法密集型的自动化开发流水线，您将能够稳定、高效且源源不断地输出在数学上绝对严谨可解，且在游玩体验上具备专业级设计水准和极高策略深度的推箱子游戏内容。

引用的著作

1. PROCEDURAL GENERATION OF SOKOBAN LEVELS - Ian Parberry, 访问时间为 三月 6, 2026，https://ianparberry.com/pubs/GAMEON-NA_METH_03.pdf
2. Procedural Generation of Sokoban Levels - Ian Parberry, 访问时间为 三月 6, 2026，https://ianparberry.com/techreports/LARC-2011-01.pdf
3. KORGym: A Dynamic Game Platform for LLM Reasoning Evaluation - arXiv, 访问时间为 三月 6, 2026，https://arxiv.org/html/2505.14552v1
4. GVGAI-LLM: Evaluating Large Language Model Agents with Infinite Games - arXiv, 访问时间为 三月 6, 2026，https://arxiv.org/html/2508.08501v2
5. BALROG: Benchmarking Agentic LLM and VLM Reasoning On Games - arXiv, 访问时间为 三月 6, 2026，https://arxiv.org/html/2411.13543v2
6. SokoBench: Evaluating Long-Horizon Planning and Reasoning in Large Language Models, 访问时间为 三月 6, 2026，https://arxiv.org/html/2601.20856v1
7. lmgame-Bench: LLM Evaluation in Games - Emergent Mind, 访问时间为 三月 6, 2026，https://www.emergentmind.com/topics/lmgame-bench
8. Sonnet 3.5 beats o1 in OpenAI's new $1M coding benchmark : r/ClaudeAI - Reddit, 访问时间为 三月 6, 2026，https://www.reddit.com/r/ClaudeAI/comments/1isncwf/sonnet_35_beats_o1_in_openais_new_1m_coding/
9. Why Generative AI Struggles with Sokoban: A Puzzle It Just Can't Solve - Medium,

访问时间为 三月 6, 2026, https://medium.com/@ikameron009/why-generative-ai-struggles-with-sokoban-a-puzzle-it-just-cant-solve-89618368fbc1
10. Long-Horizon Planning Degrades In Large Language Models After 25 Moves, 访问时间为 三月 6, 2026, https://quantumzeitgeist.com/models-long-horizon-planning-degrades-large/
11. Procedural Level Generation via Program Inversion - CEUR-WS.org, 访问时间为 三月 6, 2026, https://ceur-ws.org/Vol-4090/short4.pdf
12. The Procedural Generation of Interesting Sokoban Levels - UNT Digital Library, 访问时间为 三月 6, 2026, https://digital.library.unt.edu/ark:/67531/metadc801887/m2/1/high_res_d/dissertation.pdf
13. Procedural Generation of Sokoban Levels - Semantic Scholar, 访问时间为 三月 6, 2026, https://www.semanticscholar.org/paper/Procedural-Generation-of-Sokoban-Levels-Taylor-Parberry/702b8c6205a1f8b65dd5515807c792454b4ea7a2
14. Automatic Level Generation for Puzzle Games · Joys of Small Game Development, 访问时间为 三月 6, 2026, https://abagames.github.io/joys-of-small-game-development-en/procedural/puzzle_level.html
15. (PDF) Data-Driven Sokoban Puzzle Generation with Monte Carlo Tree Search, 访问时间为 三月 6, 2026, https://www.researchgate.net/publication/311706278_Data-Driven_Sokoban_Puzzle_Generation_with_Monte_Carlo_Tree_Search
16. Data-Driven Sokoban Puzzle Generation with Monte Carlo Tree Search, 访问时间为 三月 6, 2026, https://motion.cs.umn.edu/pub/SokobanMCTS/DataDrivenSokobanMCTS.pdf
17. Data Driven Sokoban Puzzle Generation with Monte Carlo Tree Search | Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 访问时间为 三月 6, 2026, https://ojs.aaai.org/index.php/AIIDE/article/view/12859
18. Data-Driven Sokoban Puzzle Generation with Monte Carlo Tree Search, 访问时间为 三月 6, 2026, https://cdn.aaai.org/ojs/12859/12859-52-16375-1-2-20201228.pdf
19. Difficulty Rating of Sokoban Puzzle - FI MUNI, 访问时间为 三月 6, 2026, https://www.fi.muni.cz/~xpelanek/publications/stairs2010-final.pdf
20. Introduction - Festival Sokoban Solver, 访问时间为 三月 6, 2026, https://festival-solver.site/wp-content/uploads/2020/08/FESS_draft.pdf
21. Human Problem Solving: Sokoban Case Study - FI MUNI, 访问时间为 三月 6, 2026, https://www.fi.muni.cz/reports/files/2010/FIMU-RS-2010-01.pdf
22. (PDF) Difficulty Rating of Sokoban Puzzle. - ResearchGate, 访问时间为 三月 6, 2026, https://www.researchgate.net/publication/220774696_Difficulty_Rating_of_Sokoban_Puzzle
23. Festival Sokoban Solver, 访问时间为 三月 6, 2026, https://festival-solver.site/
24. Sokoban solver "scribbles" by Florent Diedler about the Sokolution solver, 访问时

间为 三月 6, 2026, http://sokobano.de/wiki/index.php?title=Sokoban_solver_%22scribbles%22_by_Florent_Diedler_about_the_Sokolution_solver
25. Sokoban solver "scribbles" by Brian Damgaard about the YASS solver, 访问时间为 三月 6, 2026, http://sokobano.de/wiki/index.php?title=Sokoban_solver_%22scribbles%22_by_Brian_Damgaard_about_the_YASS_solver
26. Sokoban Game solving using AI Algorithms (DFS, BFS, A*, etc) - GitHub, 访问时间为 三月 6, 2026, https://github.com/rosasbehoundja/sokoban-game
27. This repository contains levels for boxoban, a box-pushing puzzle game inspired by Sokoban. - GitHub, 访问时间为 三月 6, 2026, https://github.com/google-deepmind/boxoban-levels
28. Microban Levels for Sokoban 1.x - ticalc.org, 访问时间为 三月 6, 2026, https://www.ticalc.org/archives/files/fileinfo/198/19846.html
29. Sokoban Solver Statistics - XSokoban, 访问时间为 三月 6, 2026, https://sokoban-solver-statistics.sourceforge.io/statistics/XSokoban/
30. Sokoban Utilities - 推箱子, 访问时间为 三月 6, 2026, https://sokoban.cn/utility/utilities.php
31. dangarfield/sokoban-solver: A modern, responsive web-based Sokoban puzzle game with an intelligent solver, level editor, and solution visualizer. Play classic push-the-box puzzles or let the AI solve them for you! - GitHub, 访问时间为 三月 6, 2026, https://github.com/dangarfield/sokoban-solver
32. joriswit/YASS: Yet Another Sokoban Solver and Optimizer - for Android - GitHub, 访问时间为 三月 6, 2026, https://github.com/joriswit/YASS
33. Rolling Stone solver - Sokoban Wiki, 访问时间为 三月 6, 2026, http://sokobano.de/wiki/index.php?title=Rolling_Stone_solver
34. jkl1213/A-Sokoban-Solver-Using-BFS: In this fun project I implemented Breadth First Search using Python to solve a "Push the Box" game that is quite popular on mobile. - GitHub, 访问时间为 三月 6, 2026, https://github.com/jkl1213/A-Sokoban-Solver-Using-BFS
35. ba-sokoban-solver - Niklas Sandhu Peters - GitLab, 访问时间为 三月 6, 2026, https://git.tu-berlin.de/niklassandhu/ba-sokoban-solver/-/tree/main?ref_type=heads
36. Sokoban puzzle generator and solver with BFS, A* and Dijkstra - GitHub, 访问时间为 三月 6, 2026, https://github.com/xbandrade/sokoban-solver-generator
37. A Sokoban solver - Blog of smsxgz, 访问时间为 三月 6, 2026, https://smsxgz.github.io/post/game_solver/sokoban/
38. Interpreting Multi-objective Evolutionary Algorithms via Sokoban Level Generation - arXiv, 访问时间为 三月 6, 2026, https://arxiv.org/abs/2406.10663
39. Interpreting Multi-objective Evolutionary Algorithms via Sokoban Level Generation, 访问时间为 三月 6, 2026, https://arxiv.org/html/2406.10663v1
40. Development of a Sokoban level, 访问时间为 三月 6, 2026, https://sokoban.dk/articles/1596-2/